

SUPPLEMENTARY SEMESTER EXAMINATIONS – SEPTEMBER 2025

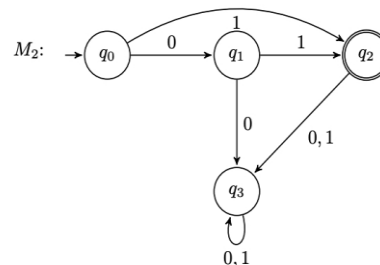
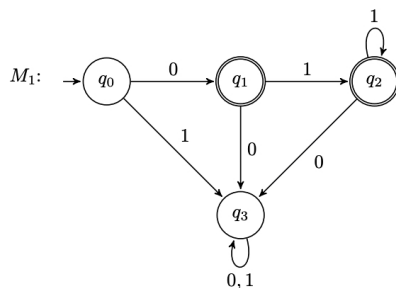
Program	: B.E. – CSE (Cyber Security)	Semester	: IV
Course Name	: Automata Theory and Compiler Design	Max. Marks	: 100
Course Code	: 22CY44	Duration	: 3 Hrs.

Instructions to the Candidates:

- Answer one full question from each unit.

UNIT - I

- Construct a NFA to recognize the strings 1101,101,111 for the $\Sigma=\{0,1\}$. CO1 (05)
 - Discuss how does a compiler process the following assignment statement CO1 (08)
total = marks * 0.8 + bonus;
 - Differentiate deterministic and non-deterministic finite automata. Also, CO1 (07)
design a deterministic and a non-deterministic finite automaton for $\Sigma=\{0,1\}$ that accepts 00 and 11 at the end of a string.
- How does the layered architecture of a language-processing system CO1 (05)
facilitate cross-platform execution, modular compilation, and debugging capabilities in contemporary software development?
 - Consider the following two DFAs (deterministic finite automata) with CO1 (06)
 $\Sigma = \{0, 1\}$:
Give the formal definition for M_1 and M_2 .



- Construct a DFA which accepts all strings over $\Sigma=\{0,1\}$, where CO1 (09)
 - even 0's and even 1's
 - accepts language $0(0|1)^*1$
 - set of strings beginning with 101.

UNIT - II

- Consider the following DFA. CO4 (10)
 $E = (Q, \Sigma, \delta, q_0, F)$, $Q = \{p_1, p_2, p_3\}$, $\Sigma = \{0, 1\}$, $q_0 = p_1$, $F = \{p_3\}$
 and δ is as follows:
 $\delta(p_1, 0) = p_2, \delta(p_1, 1) = p_1, \delta(p_2, 0) = p_3, \delta(p_2, 1) = p_1, \delta(p_3, 0) = p_3, \delta(p_3, 1) = p_2$.
 Convert the above DFA to regular expression. Construct the transition table.
 - "Assume that all our Transition diagrams are deterministic". Justify your CO1 (10)
answer for this statement.
Construct the transition diagram for the given tokens
 - logical operators
 - while, which, what, wait.

4. a) Explain the role of lexical analyzer with a neat diagram. CO2 (10)
b) Identify the token, pattern and lexeme for the given code fragment by illustrating sentinels technique. CO1 (10)

```
int main(){  
    // usage of printf statement  
    printf("i = %d, &i = %d", i, &i);  
}
```

UNIT - III

5. a) Consider the grammar CO3 (09)
G:- $S \rightarrow xABz$
 $A \rightarrow Ayw$
 $A \rightarrow y$
 $B \rightarrow u$

- i) Construct the LR (0) set of items by indicating the Kernel and Non-kernel items for each item set.
ii) Construct LR(0) parse table.
iii) Show the actions of a parser on input xyuz\$

- b) Consider the following grammar: CO3 (06)

$S \rightarrow SS+ | SS^* | a$

Input string: $aaa^*a++ \$$

Indicate the configurations of a shift-reduce parser on the above input. In the case of a reduce action, indicate which production is used. With each action, indicate whether there is conflict or not, if exists specify the type of conflict.

- c) Give the formal definition of CFG. Give CFG that generates the strings for the Language $L = \{0^n 1^n \mid n \geq 1\}$. CO3 (05)

6. a) Consider the grammar given below. If not suitable for Predictive Parsing make necessary changes and construct predictive parsing table for the grammar given below. Check whether the grammar is LL(1). CO3 (08)

G: $A \rightarrow Bd \mid a$
 $B \rightarrow Ae \mid b \mid \epsilon$
 $C \rightarrow B \mid e$

- b) Prove that the given Context Free Grammar is LR(1) by generating the parse table. CO3 (07)

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Show the actions made by the parser on validating the input string 'ab\$'.

- c) Explain the need of Augmentation in LR grammars and how the given grammar can be changed to an augmented grammar? CO3 (05)

UNIT- IV

7. a) Describe the language of a Turing Machine. Generate Turing Machine for accepting language $0^n 1^{2n}$, $\forall n \geq 1$. CO4 (10)

- b) Write L-attributed Definition for the given CFG to convert binary to decimal for input 1101. Draw an annotated parse tree for the same. CO5 (10)

G: $D \rightarrow BD^1$
 $D^1 \rightarrow BD_1^1 \mid \epsilon$
 $B \rightarrow 0 \mid 1$

8. a) Write a L-Attributed SDD used for Simple desk calculator. Obtain the Semantic action, Annotated parse tree and dependency Graph for the string $3*5+4n$. CO5 (10)

b) Consider the following attribute grammar:

CO5 (10)

Production	Semantic Rule
$S \rightarrow L_1.L_2$	$S.v = L_1.v + \frac{L_2.v}{2^{L_2.c}}$
$S \rightarrow L$	$S.v = L.v$
$L \rightarrow L_1 B$	$L.v = L_1.v * 2 + B.v$ $L.c = L_1.c + B.c$
$L \rightarrow B$	$L.v = B.v$ $L.c = B.c$
$B \rightarrow 0$	$B.v = 0$ $B.c = 1$
$B \rightarrow 1$	$B.v = 1$ $B.c = 1$

- Draw the Annotated parse tree for the input '110.01', and show the dependency graph for the associated attributes.
- Describe one correct order for the evaluation of the attributes. Assume 'c' and 'v' are the synthesized attributes.

What will be the value of S.v when evaluation has terminated?

UNIT - V

9. a) Generate a postfix SDT for the given grammar and Implement the parser stack for converting binary to decimal. If a binary value '110' is given, N.val should print the final decimal equivalent value 6. CO5 (06)

$N \rightarrow D$

$D \rightarrow D_1 B \mid B$

$B \rightarrow 0 \mid 1$

- b) Design a L- Attributed Definition for converting a binary to decimal value. CO5 (06)
Consider the Production:

G:- $B \rightarrow N D$

$D \rightarrow N D \mid \epsilon$

$N \rightarrow 0 \mid 1$

Also construct the Annotated Parse tree for input string "1101"
Hint: Binary value:= 1101 should be converted to Decimal value 13.

- c) Consider the rules given below CO5 (04)

Production	Semantic Rules
$A \rightarrow B C$	$A.s = B.b$ $B.i = f(C.c, A.s)$

Justify the following

- Is this SDD L-Attributed?
- Is B.i valid inherited attribute definition?

- d) Write the three address code for the given statements CO5 (04)

i. $f = \min(1, n-1, n+1)$

ii. $a = x[i] + b * c$

10. a) Discuss the issues in the design of a code generator. CO4 (06)

- b) Translate the following arithmetic expressions into DAG, Three address code and identify the value numbers for the following expressions, assuming operators associates from the left.(All the variables are integers) CO5 (08)

i. $(a-b)*(c+d)+(a-b)$

ii. $(a+b)*(c+d)-(a+b+c)$

iii. $(a+b)+(a+b+a+b+(a+b))$

iv. $a[i] = b * c - b * d + b * c$

- c) Illustrate the Translation Scheme using Backpatching for control flow statements. Discuss the three functions used for manipulating list of jumps in the process of translation. CO5 (06)
